Ex no: 1	Addition of Two 32-bit Numbers using ARM Cortex-M Assembly
Date:	

Aim:

To write and execute an ARM Cortex-M assembly program to add two 32-bit numbers and store the result in memory.

Code:

//01_add_32bit.s

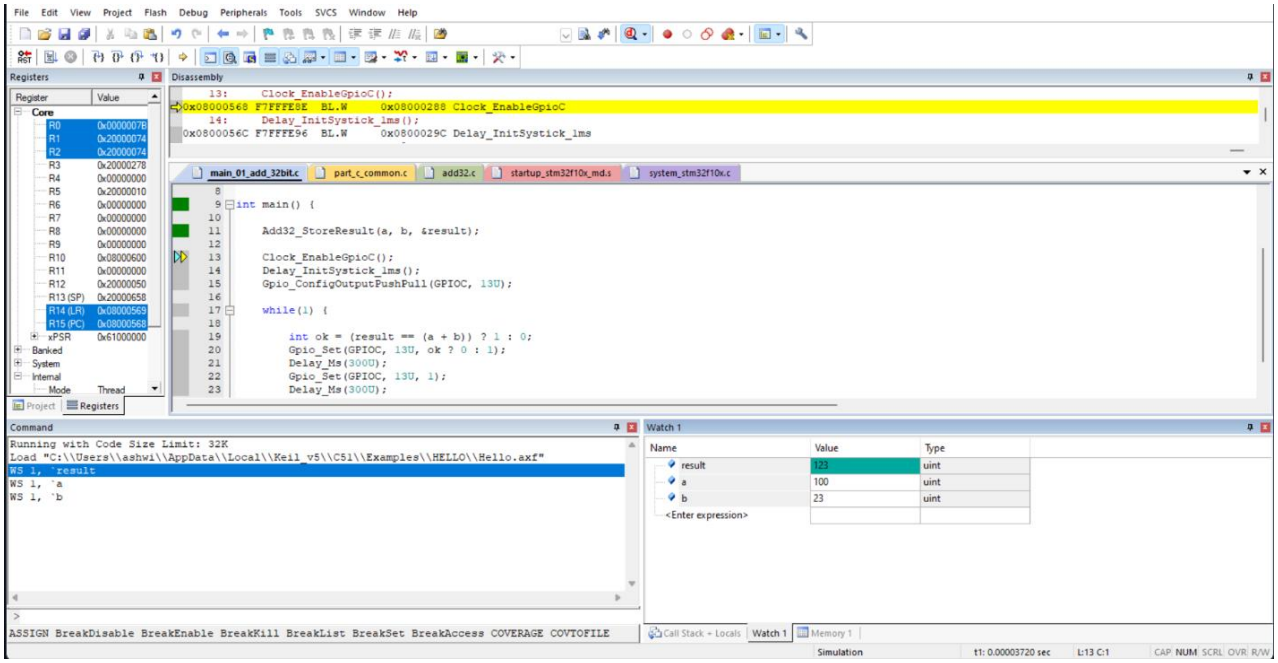
```
    AREA    |.text|, CODE, READONLY
      THUMB
      EXPORT  Add32_StoreResult
Add32_StoreResult
      ADDS    R0, R0, R1
      STR     R0, [R2]
      BX      LR
      END
```

//Main_01_add_32bit.c:

```
#include "part_c_common.h"
void Add32_StoreResult(unsigned int a, unsigned int b, unsigned int *out);
int main() {
   unsigned int a = 100U;
   unsigned int b = 23U;
   unsigned int result = 0U;
   Add32_StoreResult(a, b, &result);
   Clock_EnableGpioC();
   Delay_InitSystick_1ms();
   Gpio_ConfigOutputPushPull(GPIOC, 13U);

   while(1) {
      int ok = (result == (a + b)) ? 1 : 0;
```

```
        Gpio_Set(GPIOC, 13U, ok ? 0 : 1);
        Delay_Ms(300U);
        Gpio_Set(GPIOC, 13U, 1);
        Delay_Ms(300U);
    }
}
```

Output:



Result:

The two 32-bit numbers were added successfully and the result was stored in the specified memory location.

Ex no: 2	Fibonacci Series Generation using ARM Cortex-M Assembly
Date:	

Aim:

To develop an ARM Cortex-M assembly program to generate Fibonacci series up to 10 terms and store them in RAM.

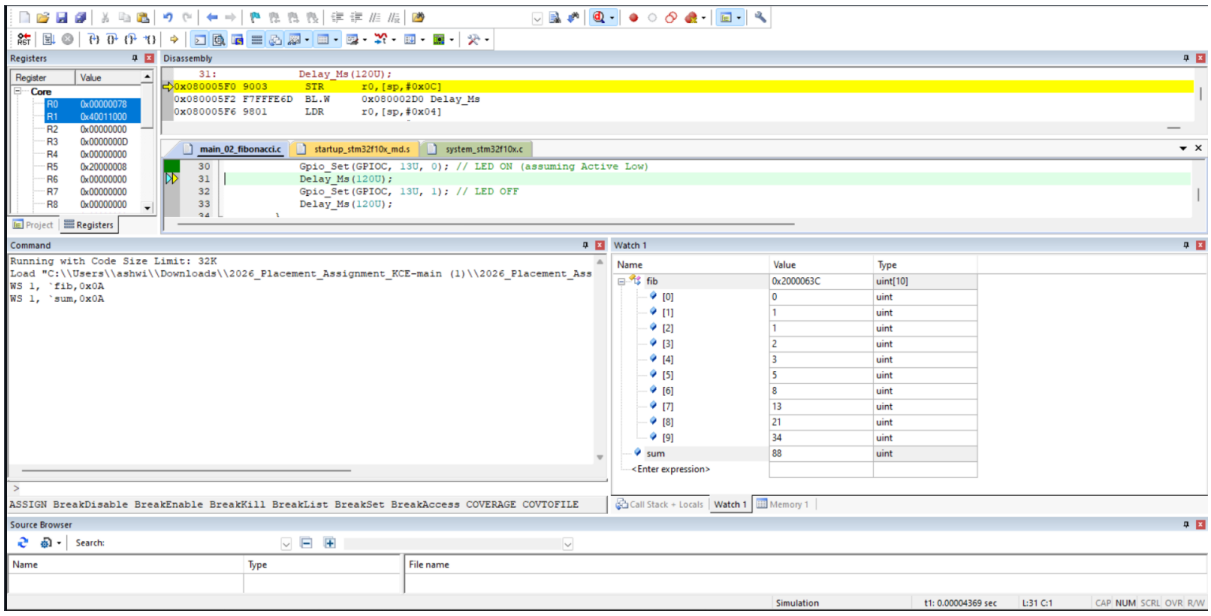
Code:

//02_fibonacci_10_terms.s

```
        AREA    |.text|, CODE, READONLY
        THUMB
        EXPORT  Fib10_Fill
Fib10_Fill
        MOVS    R1, #0
        STR     R1, [R0, #0]
        MOVS    R2, #1
        STR     R2, [R0, #4]
        MOVS    R3, #2
loop
        LDR     R1, [R0, R3, LSL #2]
        SUBS    R4, R3, #1
        LDR     R1, [R0, R4, LSL #2]
        SUBS    R5, R3, #2
        LDR     R2, [R0, R5, LSL #2]
        ADDS    R1, R1, R2
        STR     R1, [R0, R3, LSL #2]
        ADDS    R3, R3, #1
        CMP     R3, #10
        BLT     loop
        BX      LR
```

```
        END
//main_02_fibonacci_10_terms.c
#include "part_c_common.h"

void Fib10_Fill(unsigned int *out10);
int main() {
    unsigned int fib[10];
    unsigned int sum = 0U;
    Fib10_Fill(fib);
    for (unsigned int i = 0U; i < 10U; i++) sum += fib[i];
    Clock_EnableGpioC();
    Delay_InitSystick_1ms();
    Gpio_ConfigOutputPushPull(GPIOC, 13U);
    while(1) {
        unsigned int blinks = sum % 10U;
        if (blinks == 0U) blinks = 1U;
        for (unsigned int i = 0U; i < blinks; i++) {
            Gpio_Set(GPIOC, 13U, 0);
            Delay_Ms(120U);
            Gpio_Set(GPIOC, 13U, 1);
            Delay_Ms(120U);
        }
        Delay_Ms(600U);
    }
}
```

Output:



Result:

The Fibonacci series was generated correctly and the values were stored sequentially in RAM.

Ex no: 3	GPIO Pin Toggling using ARM Cortex-M Assembly
Date:	

Aim:

To write an ARM Cortex-M assembly program to toggle a GPIO pin using bit manipulation instructions.

Code:

//03_gpio_bits_toggle.s

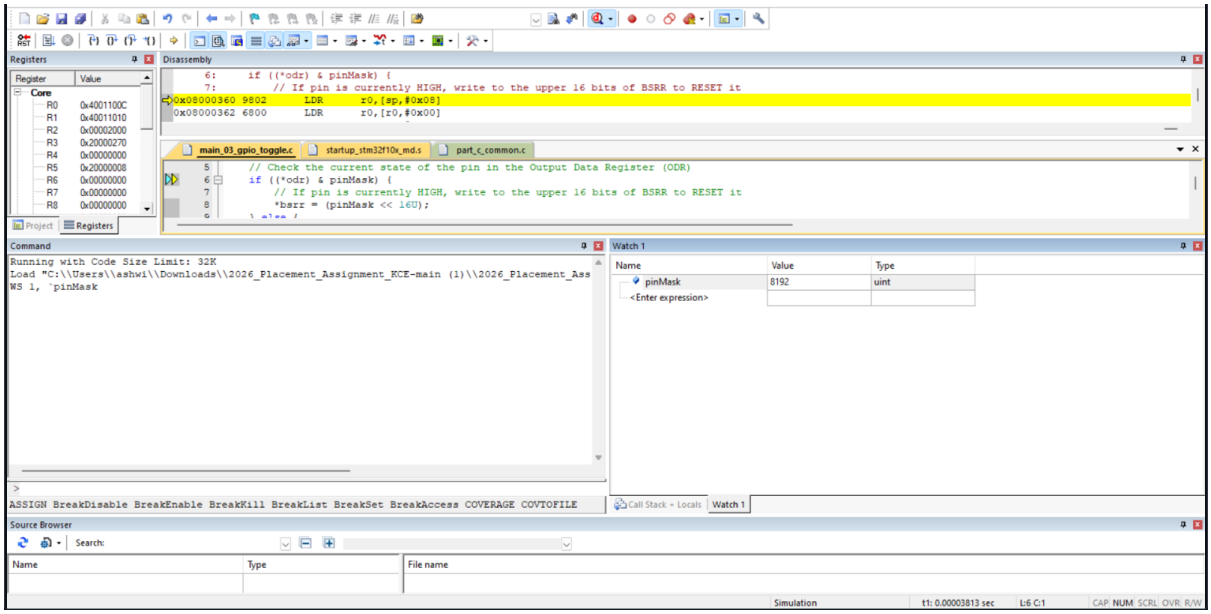
```
    AREA    |.text|, CODE, READONLY
    THUMB
    EXPORT  Gpio_Toggle_BSRR
Gpio_Toggle_BSRR
    LDR     R3, [R0]
    TST     R3, R2
    BEQ     setpin
    LSLS    R2, R2, #16
    STR     R2, [R1]
    BX      LR
setpin
    STR     R2, [R1]
    BX      LR
    END
```

//Main_03_gpio_bits_toggle.c

```
#include "part_c_common.h"

void Gpio_Toggle_BSRR(unsigned int *odr, unsigned int *bsrr, unsigned int pinMask);
```

```
int main() {
    Clock_EnableGpioC();
    Delay_InitSystick_1ms();
    Gpio_ConfigOutputPushPull(GPIOC, 13U);
    while(1) {
        Gpio_Toggle_BSRR((unsigned int *)&GPIOC->ODR, (unsigned int *)&GPIOC->BSRR, (1U << 13U));
        Delay_Ms(250U);
    }
}
```

Output:



Result:

The GPIO pin was toggled successfully and the connected LED blinked continuously.

Ex no: 4	Delay Loop Implementation using ARM Cortex-M Assembly
Date:	

Aim:

To implement a delay loop in ARM Cortex-M assembly using registers and branch instructions.

Code:

//04_delay_loop.s

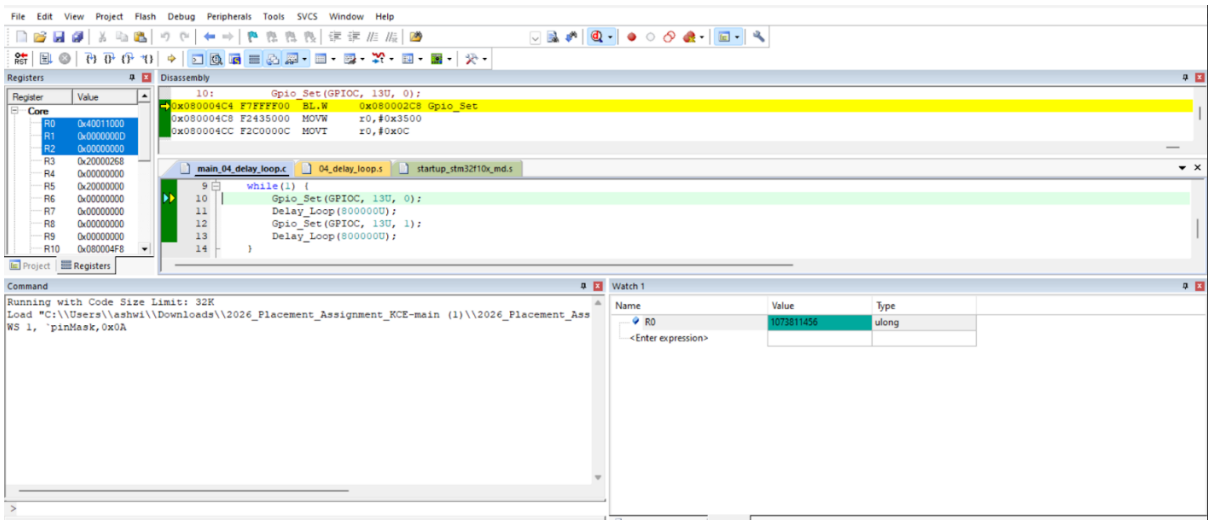
```
        AREA    |.text|, CODE, READONLY
        THUMB
        EXPORT  Delay_Loop
Delay_Loop
        CMP     R0, #0
        BEQ     done
loop
        SUBS    R0, R0, #1
        BNE     loop
done
        BX      LR
        END
```

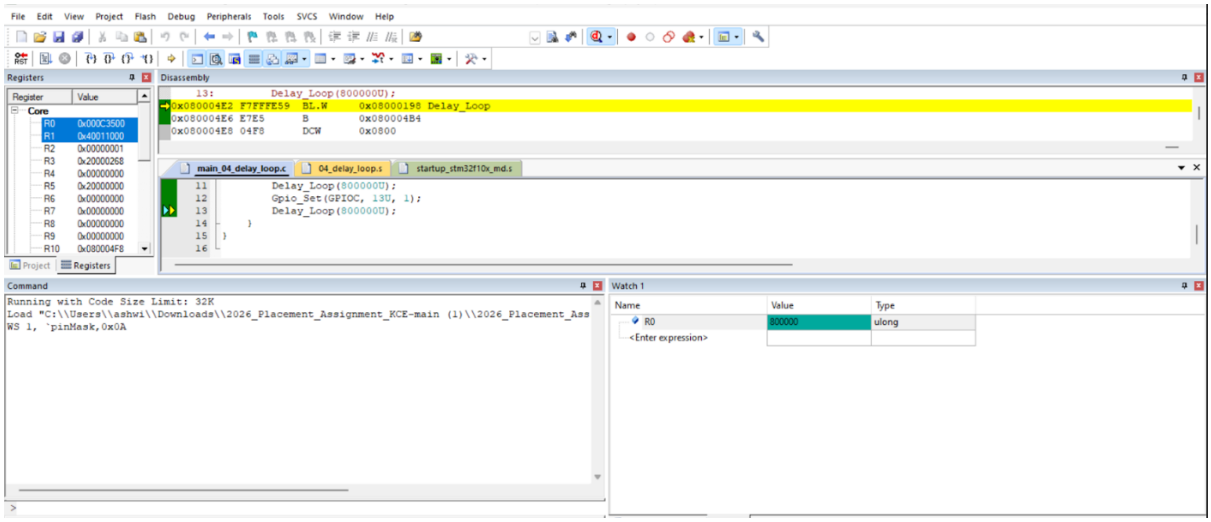
//main_04_delay_loop.c

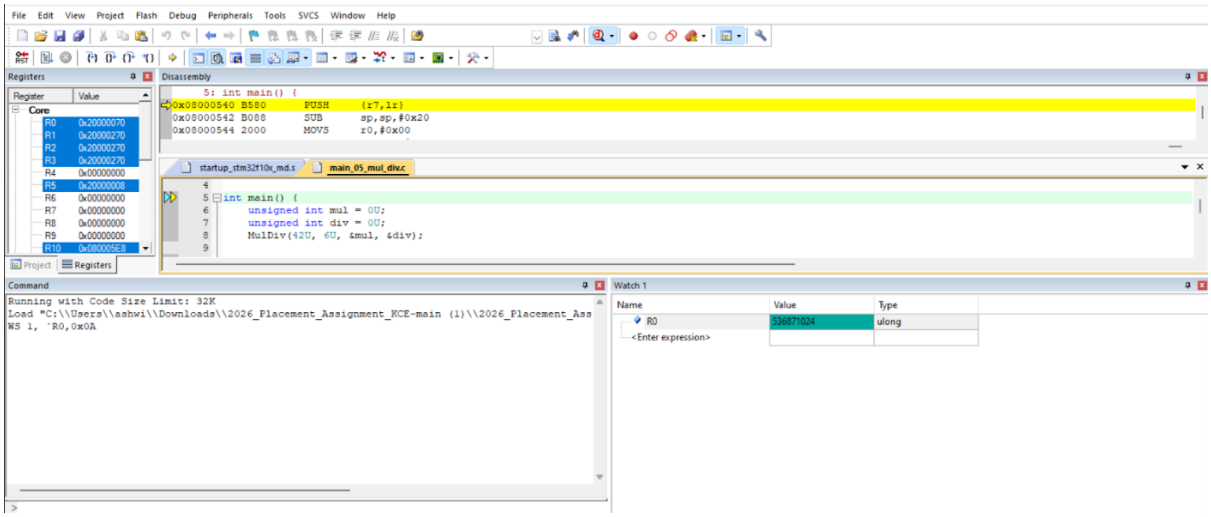
```
#include "part_c_common.h"
void Delay_Loop(unsigned int count);
int main() {
    Clock_EnableGpioC();
    Gpio_ConfigOutputPushPull(GPIOC, 13U);
    while(1) {
        Gpio_Set(GPIOC, 13U, 0);
        Delay_Loop(800000U);
```

```
        Gpio_Set(GPIOC, 13U, 1);
        Delay_Loop(800000U);
    }
}
```

Output:







Result:

A required time delay was generated successfully using looping instructions.

Ex no: 5	Multiplication And Division Using Registers
Date:	

Aim:

To create an ARM Cortex-M assembly program to perform multiplication and division of two numbers using registers.

Code:

//05_mul_div.s

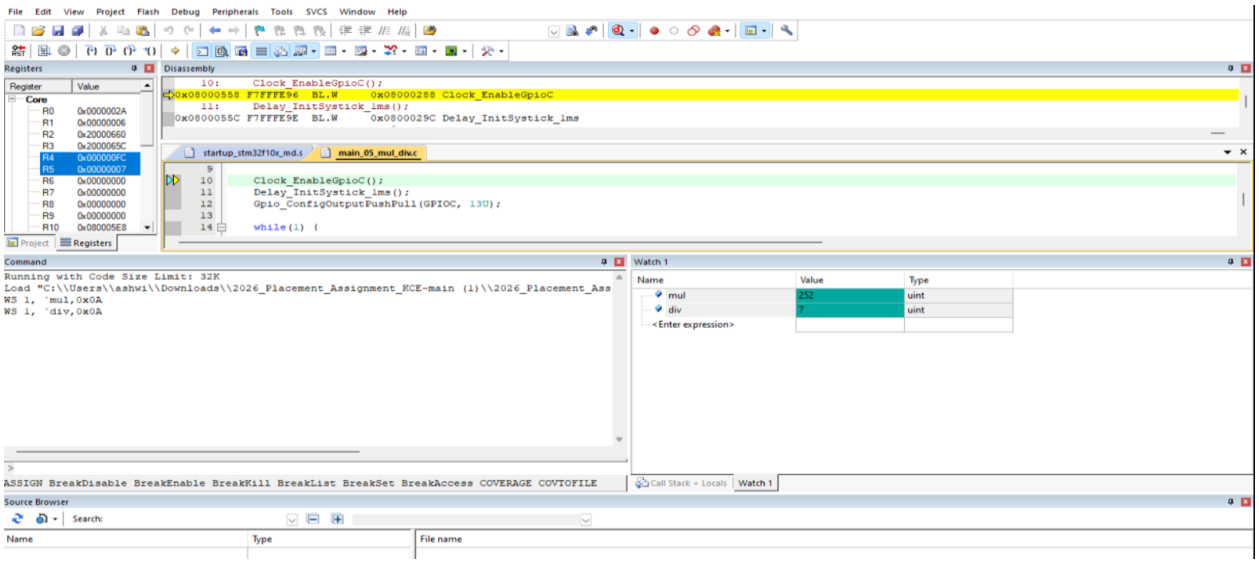
```
        AREA    |.text|, CODE, READONLY
        THUMB
        EXPORT  MulDiv
MulDiv
        MUL     R4, R0, R1
        STR     R4, [R2]
        CMP     R1, #0
        BEQ     div_zero
        UDIV    R5, R0, R1
        STR     R5, [R3]
        BX      LR
div_zero
        MOVS    R5, #0
        STR     R5, [R3]
        BX      LR
        END
```

//Main_mul_div.c

```
#include "part_c_common.h"

void MulDiv(unsigned int a, unsigned int b, unsigned int *mulOut, unsigned int *divOut);
```

```
int main() {
    unsigned int mul = 0U;
    unsigned int div = 0U;
    MulDiv(42U, 6U, &mul, &div);
    Clock_EnableGpioC();
    Delay_InitSystick_1ms();
    Gpio_ConfigOutputPushPull(GPIOC, 13U);

    while(1) {
        int ok = (mul == 252U) && (div == 7U);
        Gpio_Set(GPIOC, 13U, ok ? 0 : 1);
        Delay_Ms(150U);
        Gpio_Set(GPIOC, 13U, 1);
        Delay_Ms(450U);
    }
}
```

Output:



Result:

Multiplication and division operations were performed successfully and correct results were obtained.

Ex no: 6	LED Interfacing And Blinking Using GPIO in C
Date:	

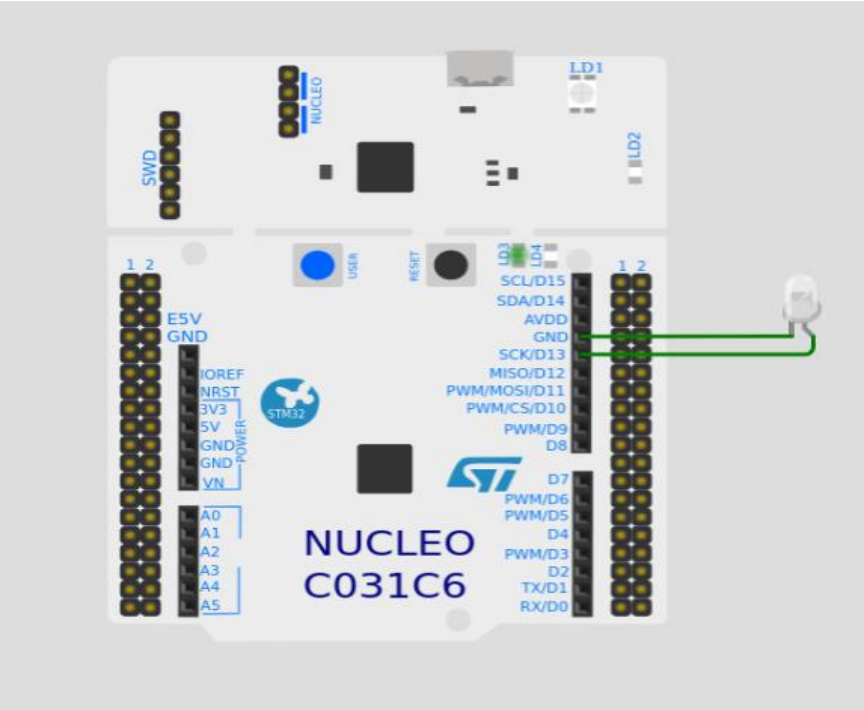
Aim:

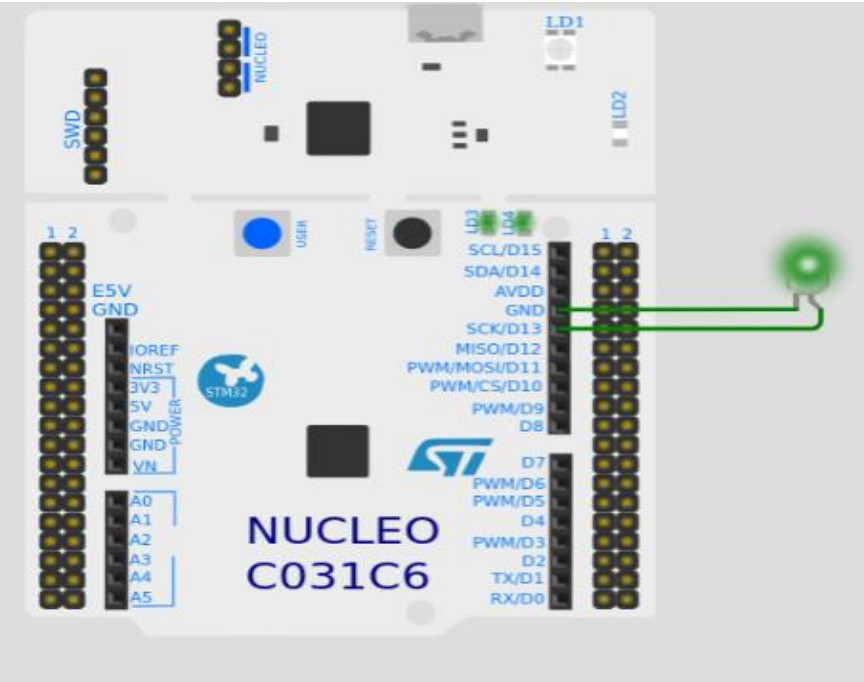
To interface an LED with an ARM Cortex board and write a C program to blink the LED using GPIO.

Code:

```
#include "part_c_common.h"

int main() {

    Clock_EnableGpioC();

    Delay_InitSystick_1ms();

    Gpio_ConfigOutputPushPull(GPIOC, 13U);

    while(1) {

        Gpio_Set(GPIOC, 13U, 0);

        Delay_Ms(300U);

        Gpio_Set(GPIOC, 13U, 1);

        Delay_Ms(300U);

    }

}
```

Output:





Result:

The LED was interfaced successfully and blinked at regular intervals.

Ex no: 7	16x2 LCD Interfacing in 4-Bit Mode
Date:	

Aim:

To interface a 16×2 LCD with an ARM Cortex board and display the message “HELLO WORLD” using GPIO in 4-bit mode.

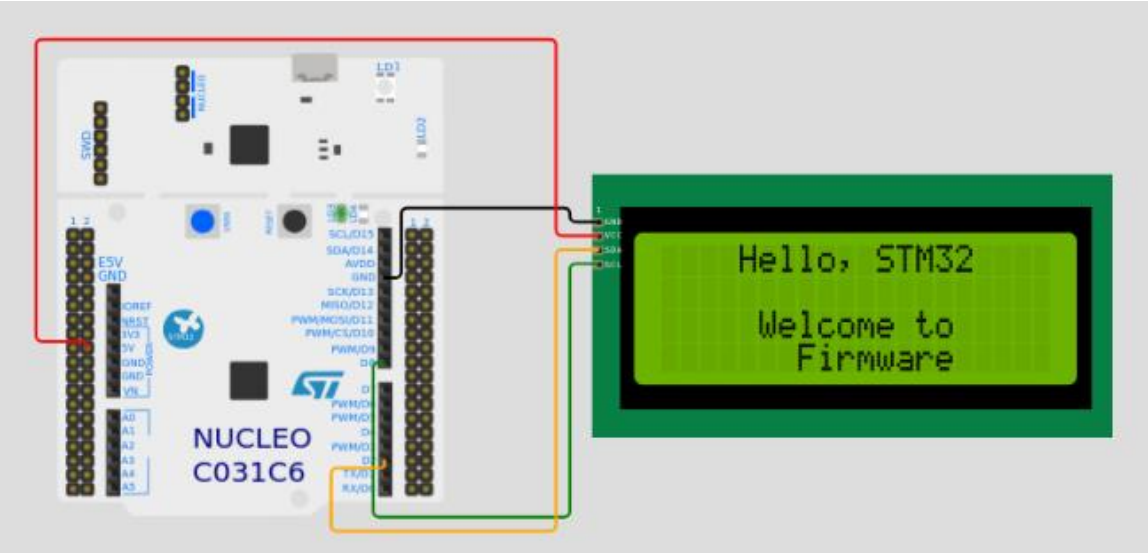
Code:

```
#include "part_c_common.h"
void Lcd_PulseEnable(void);
void Lcd_Write4(unsigned int nibble);
void Lcd_WriteCmd(unsigned int cmd);
void Lcd_WriteData(unsigned int data);
void Lcd_Init4bit(void);
void Lcd_SetCursor(unsigned int row, unsigned int col);
void Lcd_Print(const char *s);
void Lcd_PulseEnable(void) {
   Gpio_Set(GPIOB, 1U, 1);
   Delay_Ms(1U);
   Gpio_Set(GPIOB, 1U, 0);
   Delay_Ms(1U);
}
void Lcd_Write4(unsigned int nibble) {
   Gpio_Set(GPIOB, 2U, (nibble >> 0) & 1U);
   Gpio_Set(GPIOB, 10U, (nibble >> 1) & 1U);
   Gpio_Set(GPIOB, 11U, (nibble >> 2) & 1U);
   Gpio_Set(GPIOB, 12U, (nibble >> 3) & 1U);
   Lcd_PulseEnable();
}
```

```
void Lcd_WriteCmd(unsigned int cmd) {
    Gpio_Set(GPIOB, 0U, 0);
    Lcd_Write4(cmd >> 4);
    Lcd_Write4(cmd & 0x0FU);
    Delay_Ms(2U);
}
void Lcd_WriteData(unsigned int data) {
    Gpio_Set(GPIOB, 0U, 1);
    Lcd_Write4(data >> 4);
    Lcd_Write4(data & 0x0FU);
    Delay_Ms(2U);
}
void Lcd_Init4bit(void) {
    Clock_EnableGpioB();
    Delay_InitSystick_1ms();
    Gpio_ConfigOutputPushPull(GPIOB, 0U);
    Gpio_ConfigOutputPushPull(GPIOB, 1U);
    Gpio_ConfigOutputPushPull(GPIOB, 2U);
    Gpio_ConfigOutputPushPull(GPIOB, 10U);
    Gpio_ConfigOutputPushPull(GPIOB, 11U);
    Gpio_ConfigOutputPushPull(GPIOB, 12U);
    Delay_Ms(40U);
    Gpio_Set(GPIOB, 0U, 0);
    Gpio_Set(GPIOB, 1U, 0);
    Lcd_Write4(0x3U);
    Delay_Ms(5U);
    Lcd_Write4(0x3U);
    Delay_Ms(5U);
    Lcd_Write4(0x3U);
```

```
    Delay_Ms(1U);
    Lcd_Write4(0x2U);
    Lcd_WriteCmd(0x28U);
    Lcd_WriteCmd(0x0CU);
    Lcd_WriteCmd(0x06U);
    Lcd_WriteCmd(0x01U);
    Delay_Ms(5U);
}
void Lcd_SetCursor(unsigned int row, unsigned int col) {
    unsigned int addr = (row == 0U) ? 0x00U : 0x40U;
    addr += col;
    Lcd_WriteCmd(0x80U | addr);
}
void Lcd_Print(const char *s) {
    while (*s) Lcd_WriteData((unsigned int)*s++);
}

int main() {
    Lcd_Init4bit();
    Lcd_SetCursor(0U, 0U);
    Lcd_Print("HELLO WORLD");
    while(1) {}
}
```

Output:



Result:

The LCD was interfaced successfully and the required message was displayed correctly.

Ex no: 8	DC Motor Control Using GPIO
Date:	

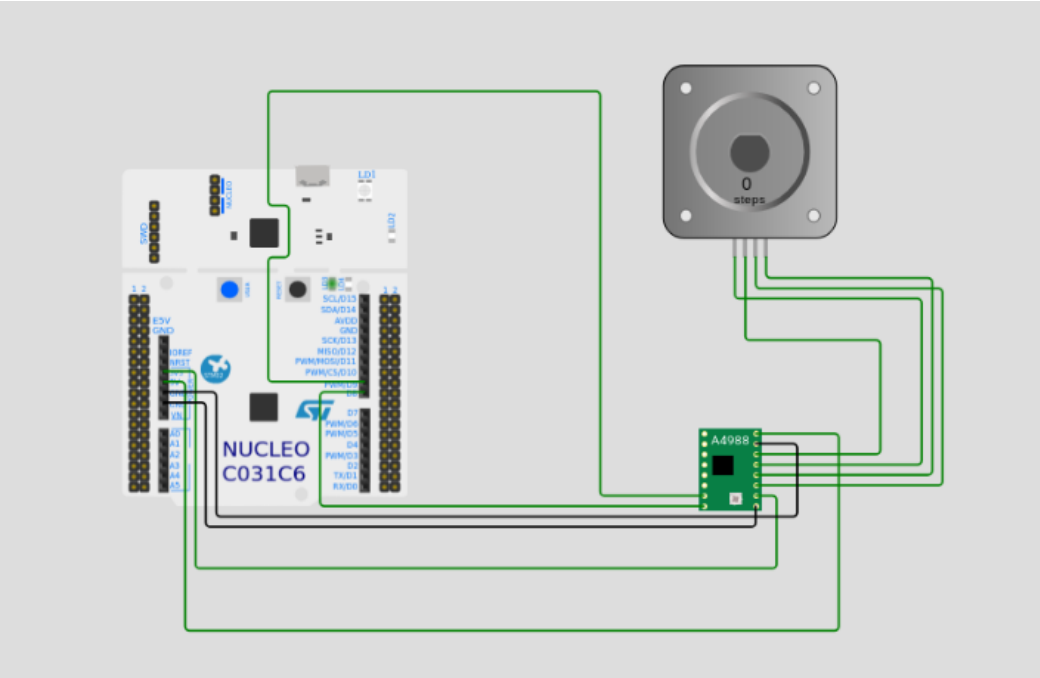
Aim:

To control a DC motor using an ARM Cortex board and change its direction in forward and reverse using GPIO signals.

Code:

```
#include "part_c_common.h"

void Motor_Set(int forward);

void Motor_Set(int forward) {

    if (forward) {

        Gpio_Set(GPIOA, 0U, 1);

        Gpio_Set(GPIOA, 1U, 0);

    } else {

        Gpio_Set(GPIOA, 0U, 0);

        Gpio_Set(GPIOA, 1U, 1);

    }

}

int main() {

    Clock_EnableGpioA();

    Delay_InitSystick_1ms();

    Gpio_ConfigOutputPushPull(GPIOA, 0U);

    Gpio_ConfigOutputPushPull(GPIOA, 1U);

    while(1) {

        Motor_Set(1);

        Delay_Ms(1000U);

        Motor_Set(0);

        Delay_Ms(1000U);

    }

}
```

Output:




```
--- STM32 Stepper Motor Controller ---
Direction: CW, Speed: 1000 Hz _ 2300049078
Direction: CW, Speed: 200 Hz _ 2300049078
Direction: CW, Motor Stopped.
Direction: CCW, Speed: 1000 Hz _ 2300049078
Direction: CCW, Motor Stopped.
Direction: CW, Speed: 1000 Hz _ 2300049078
```

Result:

The DC motor operated successfully in both forward and reverse directions.

Ex no: 9	4x4 Keypad Interfacing With LCD Display
Date:	

Aim:

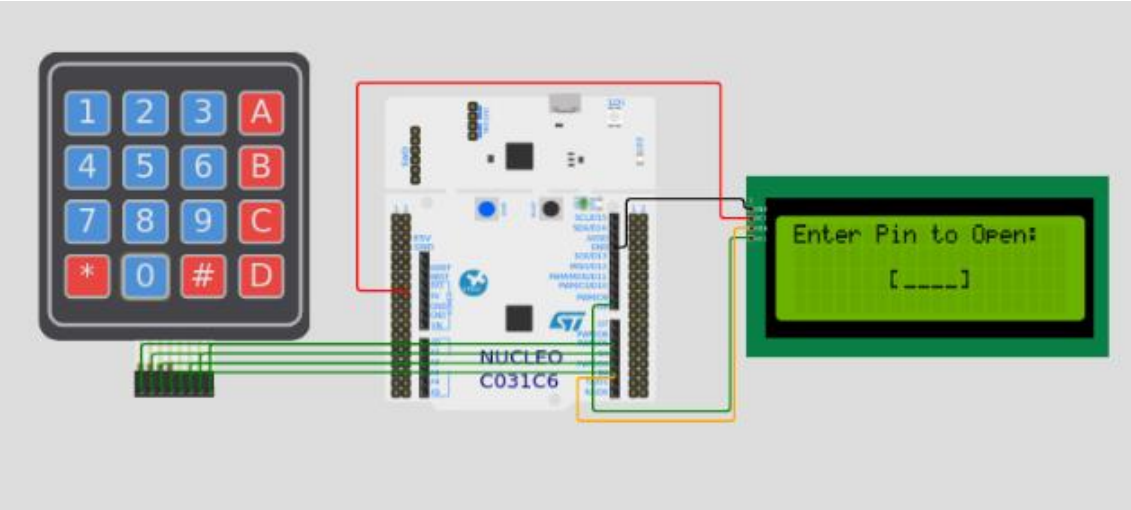
To interface a 4×4 keypad with an ARM Cortex board and display the pressed key value on an LCD.

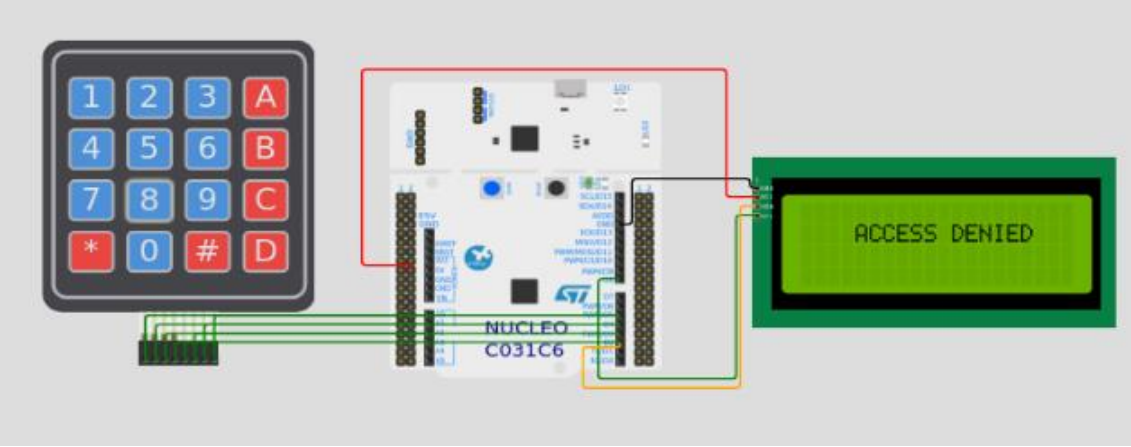
Code:

```
#include "part_c_common.h"

char Keypad_Scan(void);

char Keypad_Scan(void) {

    char map[4][4] = {

        {'1','2','3','A'},

        {'4','5','6','B'},

        {'7','8','9','C'},

        {'*','0','#','D'}

    };


    for (unsigned int r = 0U; r < 4U; r++) {

        for (unsigned int i = 0U; i < 4U; i++) Gpio_Set(GPIOA, i, 1);

        Gpio_Set(GPIOA, r, 0);

        Delay_Ms(1U);

        for (unsigned int c = 0U; c < 4U; c++) {

            if (Gpio_Get(GPIOA, 4U + c) == 0) return map[r][c];

        }

    }

    return 0;

}
```

```
int main() {
    Clock_EnableGpioA();
    Delay_InitSystick_1ms();
    Usart1_Init_9600_8N1();

    for (unsigned int i = 0U; i < 4U; i++) Gpio_ConfigOutputPushPull(GPIOA, i);

    for (unsigned int i = 4U; i < 8U; i++) Gpio_ConfigInputPull(GPIOA, i, 1);
    Usart1_WriteString("Keypad scan (press keys)\r\n");

    while(1) {
        char k = Keypad_Scan();
        if (k) {
            Usart1_WriteChar(k);
            Usart1_WriteString("\r\n");
            while (Keypad_Scan()) Delay_Ms(20U);
        }
        Delay_Ms(50U);
    }
}
```

Output:





Result:

The pressed key was detected correctly and displayed on the LCD.

Ex no: 10	Relay Interfacing For External Lamp Control
Date:	

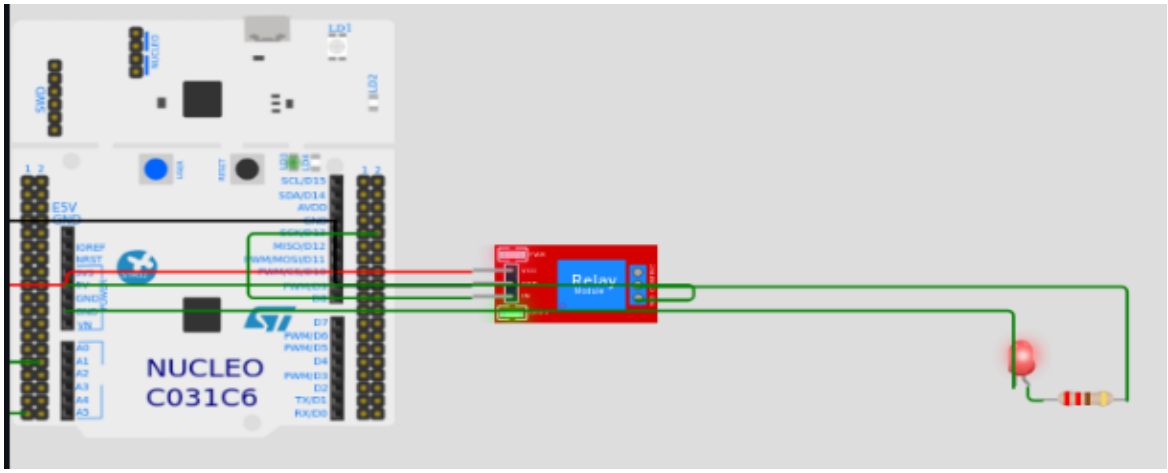
Aim:

To interface a relay with an ARM Cortex board to control an external lamp based on switch input.

Code:

```
#include "part_c_common.h"

int main() {

    Clock_EnableGpioC();

    Clock_EnableGpioA();

    Delay_InitSystick_1ms();

    Gpio_ConfigOutputPushPull(GPIOC, 13U);

    Gpio_ConfigInputPull(GPIOA, 0U, 1);

    int relay_on = 0;

    while(1) {

        int pressed = (Gpio_Get(GPIOA, 0U) == 0);

        if (pressed) {

            relay_on = !relay_on;

            Gpio_Set(GPIOC, 13U, relay_on ? 0 : 1);

            while (Gpio_Get(GPIOA, 0U) == 0) Delay_Ms(20U);

        }

        Delay_Ms(20U);

    }

}
```

Output:



Result:

The relay was operated successfully and the lamp was switched ON and OFF according to the input condition.

Ex no: 11	DHT11 Sensor Interfacing
Date:	

Aim:

To interface a DHT11 sensor with an ARM Cortex board and display temperature and humidity values on an LCD.

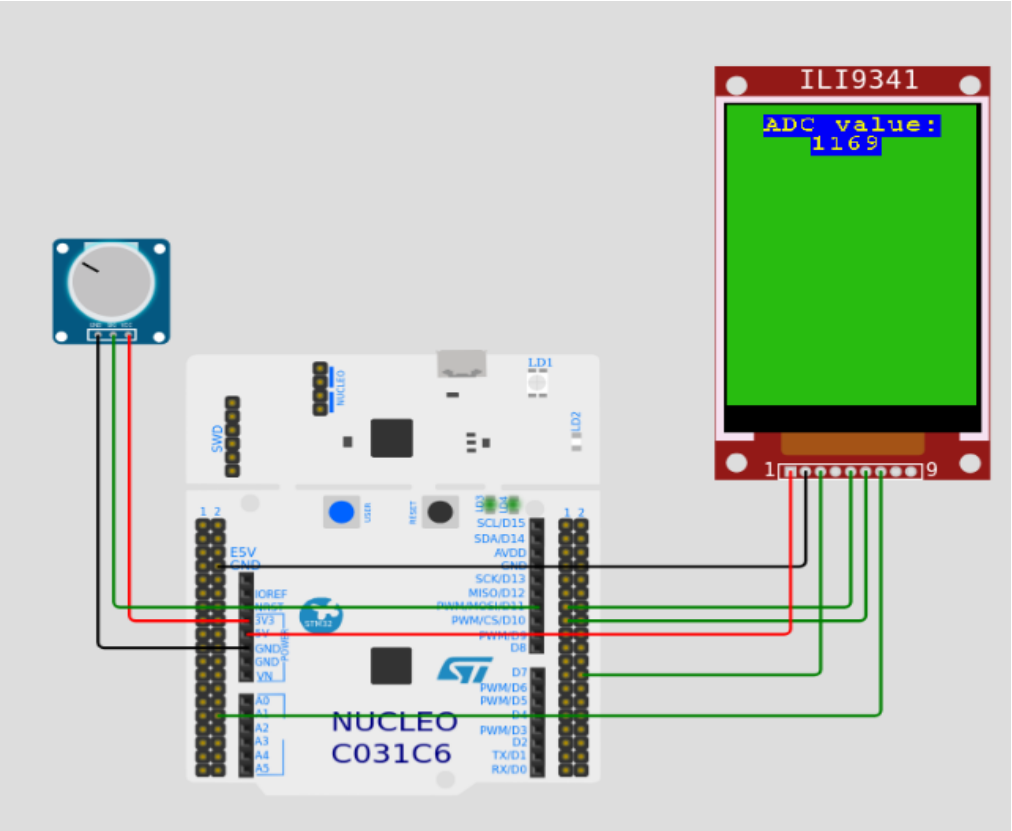
Code:

```
#include "part_c_common.h"

void Dht11_SetOutput(void);

void Dht11_SetInputPullup(void);

int  Dht11_ReadByte(unsigned int *out);

int  Dht11_Read(int *temp_c, int *humidity);

void Dht11_SetOutput(void) {

   Gpio_ConfigOutputPushPull(GPIOA, 1U);

}

void Dht11_SetInputPullup(void) {

   Gpio_ConfigInputPull(GPIOA, 1U, 1);

}


int Dht11_ReadByte(unsigned int *out) {

   unsigned int v = 0U;

   for (unsigned int i = 0U; i < 8U; i++) {

      unsigned int t = 0U;

      while (Gpio_Get(GPIOA, 1U) == 0) { if (++t > 80000U) return 0; }

      t = 0U;

      while (Gpio_Get(GPIOA, 1U) == 1) { if (++t > 80000U) break; }

      v <<= 1;

      if (t > 2000U) v |= 1U;

   }
```

```
    *out = v;
    return 1;
}

int Dht11_Read(int *temp_c, int *humidity) {
    unsigned int hI, hD, tI, tD, cs;
    Dht11_SetOutput();
    Gpio_Set(GPIOA, 1U, 0);
    Delay_Ms(20U);
    Gpio_Set(GPIOA, 1U, 1);
    Dht11_SetInputPullup();
    Delay_Ms(1U);
    unsigned int t = 0U;
    while (Gpio_Get(GPIOA, 1U) == 1) { if (++t > 80000U) return 0; }
    t = 0U;
    while (Gpio_Get(GPIOA, 1U) == 0) { if (++t > 80000U) return 0; }
    t = 0U;
    while (Gpio_Get(GPIOA, 1U) == 1) { if (++t > 80000U) return 0; }
    if (!Dht11_ReadByte(&hI)) return 0;
    if (!Dht11_ReadByte(&hD)) return 0;
    if (!Dht11_ReadByte(&tI)) return 0;
    if (!Dht11_ReadByte(&tD)) return 0;
    if (!Dht11_ReadByte(&cs)) return 0;
    if (((hI + hD + tI + tD) & 0xFFU) != (cs & 0xFFU)) return 0;
    *humidity = (int)hI;
    *temp_c = (int)tI;
    return 1;
}
```

```
int main() {
    Clock_EnableGpioA();
    Delay_InitSystick_1ms();
    Usart1_Init_9600_8N1();
    while(1) {
        int t, h;
        if (Dht11_Read(&t, &h)) {
            char buf[64];
            unsigned int n = (unsigned int)sprintf(buf, "DHT11: T=%dC H=%d%%\r\n", t, h);
            (void)n;
            Usart1_WriteString(buf);
        } else {
            Usart1_WriteString("DHT11 read failed\r\n");
        }
        Delay_Ms(1200U);
    }
}
```

Output:



Result:

Temperature and humidity values were measured successfully and displayed correctly on the LCD.

Ex no:	I2C LCD Interfacing
Date:	

Aim:

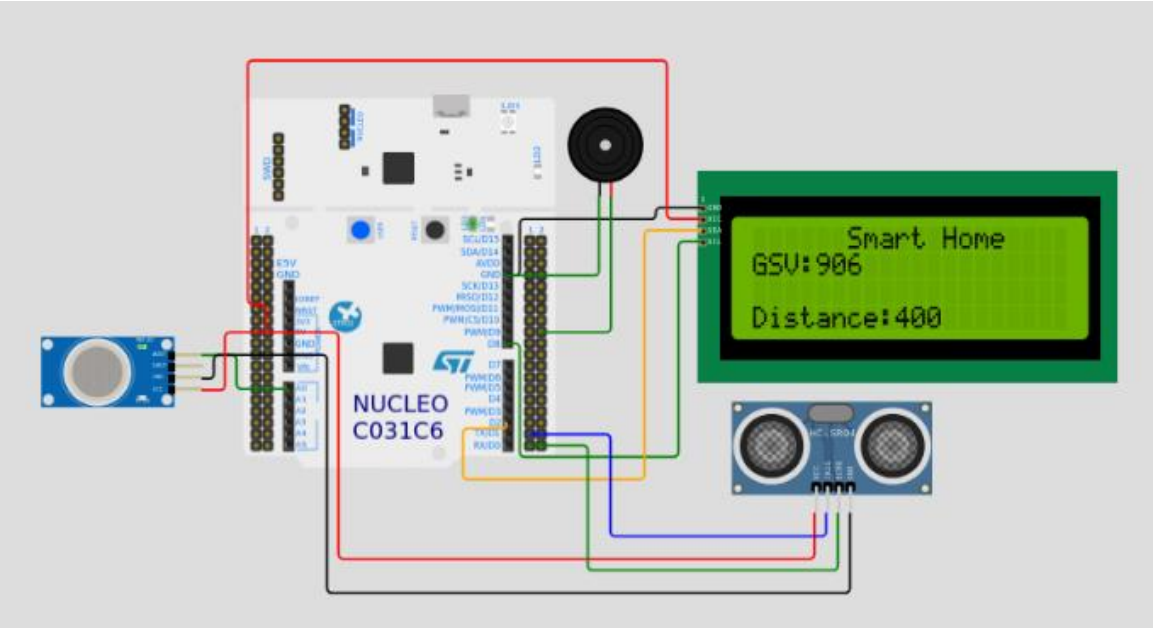
To interface an I2C LCD with an ARM Cortex board and display sensor values using I2C communication.

Code:

```
#include "part_c_common.h"

void I2c1_Init_100kHz(void);

void I2c1_WriteByte(unsigned char addr7, unsigned char data);

void I2c1_Init_100kHz(void) {

    Clock_EnableGpioB();

    RCC->APB1ENR |= RCC_APB1ENR_I2C1EN;

    {

        unsigned int v = GPIOB->CRL;

        v &= ~(0xFU << (6U * 4U));

        v |= (0xFU << (6U * 4U));

        v &= ~(0xFU << (7U * 4U));

        v |= (0xFU << (7U * 4U));

        GPIOB->CRL = v;

    }

    I2C1->CR2 = 36U;

    I2C1->CCR = 180U;

    I2C1->TRISE = 37U;

    I2C1->CR1 = I2C_CR1_PE;

}

void I2c1_WriteByte(unsigned char addr7, unsigned char data) {

    I2C1->CR1 |= I2C_CR1_START;

    while (!(I2C1->SR1 & I2C_SR1_SB)) {}
```

```
    (void)I2C1->SR1;
    I2C1->DR = (unsigned int)(addr7 << 1);
    while (!(I2C1->SR1 & I2C_SR1_ADDR)) {}
    (void)I2C1->SR1;
    (void)I2C1->SR2;
    while (!(I2C1->SR1 & I2C_SR1_TXE)) {}
    I2C1->DR = data;
    while (!(I2C1->SR1 & I2C_SR1_BTF)) {}
    I2C1->CR1 |= I2C_CR1_STOP;
}
int main() {
    Delay_InitSystick_1ms();
    I2c1_Init_100kHz();
    while(1) {
        I2c1_WriteByte(0x27U, 0x00U);
        Delay_Ms(500U);
    }
}
```

Output:



Result:

Sensor data was transmitted through I2C communication and displayed properly on the LCD.

Ex no: 13	PWM Generation Using Timer For Motor Speed Control
Date:	

Aim:

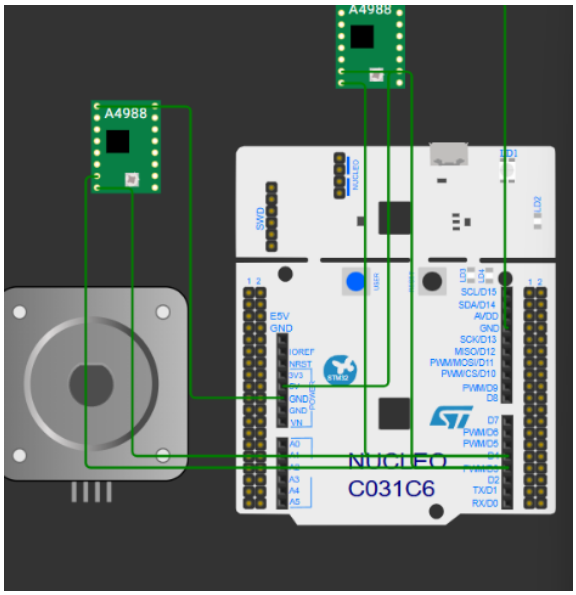
To generate a PWM signal using the Timer module of an ARM Cortex board to control the speed of a motor.

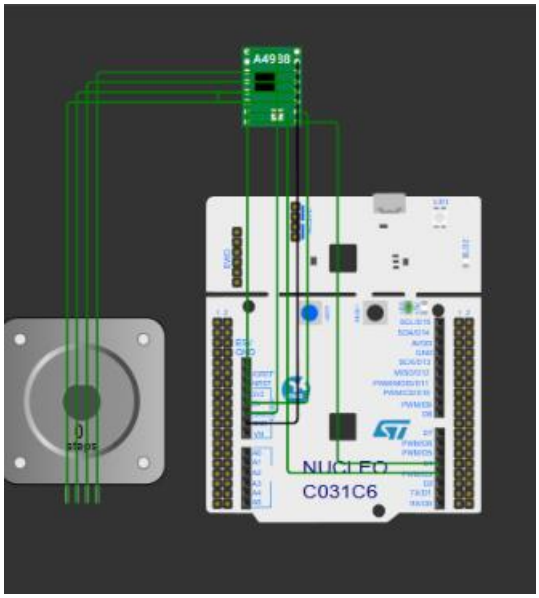
Code:

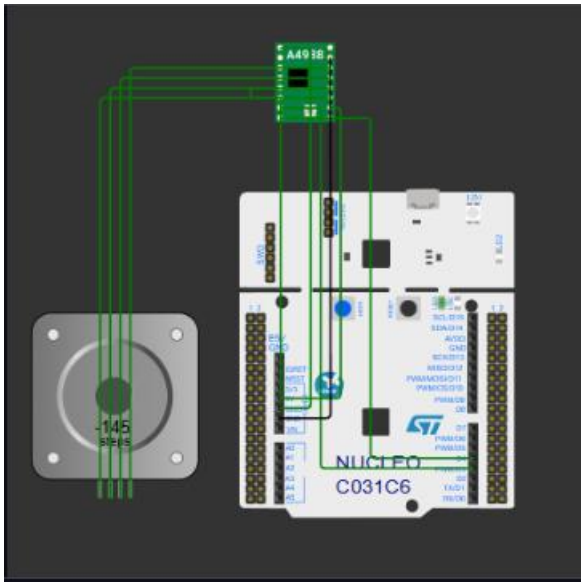
```
#include "part_c_common.h"
void Pwm_Tim2Ch1_Init(void);
void Pwm_Tim2Ch1_SetDutyPermille(unsigned int permille);
void Pwm_Tim2Ch1_Init(void) {
    Clock_EnableGpioA();
    RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
    {
        unsigned int v = GPIOA->CRL;
        v &= ~(0xFU << (0U * 4U));
        v |= (0xBU << (0U * 4U));
        GPIOA->CRL = v;
    }
    TIM2->PSC = 72U - 1U;
    TIM2->ARR = 1000U - 1U;
    TIM2->CCR1 = 0U;
    TIM2->CCMR1 = (6U << 4U) | TIM_CCMR1_OC1PE;
    TIM2->CCER = TIM_CCER_CC1E;
    TIM2->CR1 = TIM_CR1_ARPE | TIM_CR1_CEN;
}
void Pwm_Tim2Ch1_SetDutyPermille(unsigned int permille) {
    if (permille > 1000U) permille = 1000U;
    TIM2->CCR1 = permille;
```

```
}
int main() {
    Delay_InitSystick_1ms();
    Pwm_Tim2Ch1_Init();
    while(1) {
        for (unsigned int d = 0U; d <= 1000U; d += 50U) {
            Pwm_Tim2Ch1_SetDutyPermille(d);
            Delay_Ms(120U);
        }
        for (unsigned int d = 1000U; d > 0U; d -= 50U) {
            Pwm_Tim2Ch1_SetDutyPermille(d);
            Delay_Ms(120U);
        }
    }
}
```

Output:







Result:

PWM signal was generated successfully and the motor speed was controlled as required.

Ex no: 14	UART Communication Between Two ARM Cortex Boards
Date:	

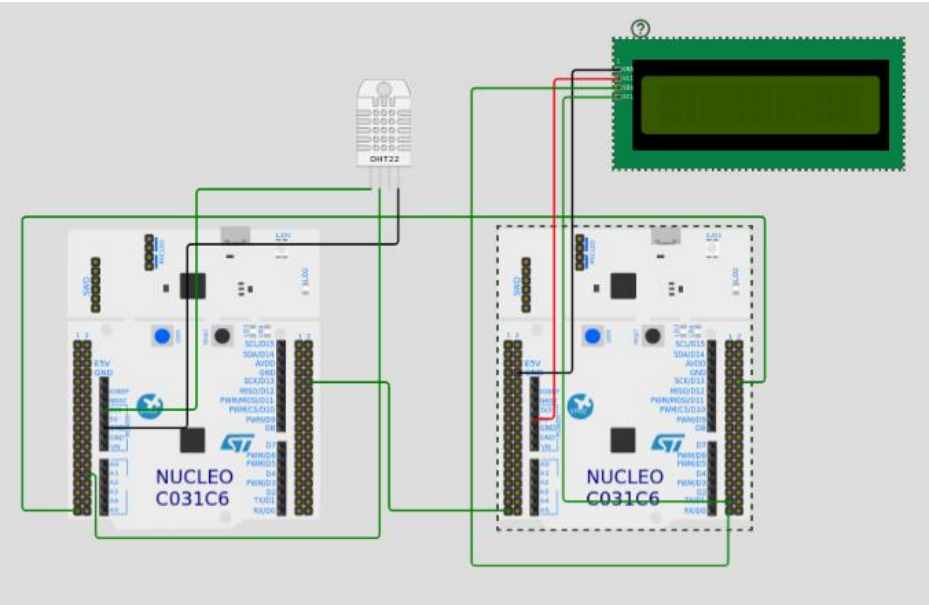
Aim:

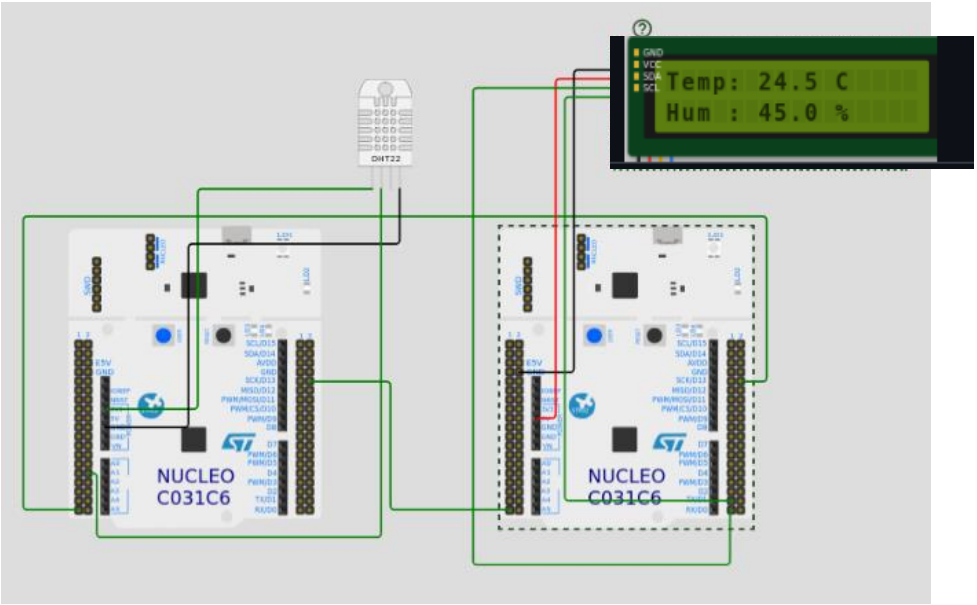
To implement UART communication to send and receive characters between two ARM Cortex boards.

Code:

```
#include "part_c_common.h"


int main() {
    Delay_InitSystick_1ms();
    Usart1_Init_9600_8N1();
    Usart1_WriteString("USART1 echo ready\r\n");
    while(1) {
        char c = Usart1_ReadCharBlocking();
        Usart1_WriteChar(c);
    }
}
```

Output:





Result:

Serial data transmission and reception were carried out successfully between the boards.

Ex no: 15	ADC Interfacing For Analog Sensor Reading
Date:	

Aim:

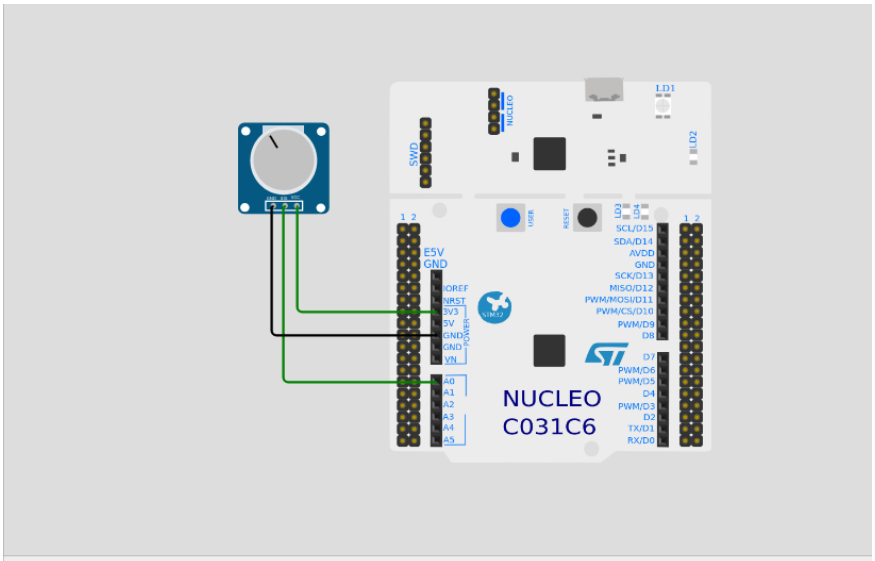
To interface an analog sensor with an ARM Cortex board and read the sensor input using ADC.

Code:

```
#include "part_c_common.h"
void Adc1_Init_Channel0(void);
unsigned int Adc1_Read_Channel0(void);
void Adc1_Init_Channel0(void) {
    Clock_EnableGpioA();
    RCC->APB2ENR |= RCC_APB2ENR_ADC1EN;
    GPIOA->CRL &= ~(0xFU << (0U * 4U));
    ADC1->CR2 = ADC_CR2_ADON;
    Delay_Ms(1U);
    ADC1->SMPR2 |= (7U << 0U);
    ADC1->SQR3 = 0U;
}
unsigned int Adc1_Read_Channel0(void) {
    ADC1->CR2 |= ADC_CR2_ADON;
    ADC1->CR2 |= ADC_CR2_SWSTART;
    while (!(ADC1->SR & ADC_SR_EOC)) {}
    return ADC1->DR;
}
int main() {
    Delay_InitSystick_1ms();
    Usart1_Init_9600_8N1();
    Adc1_Init_Channel0();
```

```
while(1) {
    unsigned int v = Adc1_Read_Channel0();
    char buf[64];
    (void)sprintf(buf, "ADC0=%u\r\n", v);
    Usart1_WriteString(buf);
    Delay_Ms(500U);
}}
```

Output:




```
STM32 Nucleo C031C6 ADC Example
Rotate potentiometer to see ADC values.
ADC: 1045       Voltage: 0.842 V
ADC: 4095       Voltage: 3.300 V
ADC: 0  Voltage: 0.000 V
ADC: 0  Voltage: 0.000 V
ADC: 0  Voltage: 0.000 V
ADC: 636        Voltage: 0.513 V
ADC: 1457       Voltage: 1.174 V
ADC: 1577       Voltage: 1.271 V
ADC: 1577       Voltage: 1.271 V
ADC: 1577       Voltage: 1.271 V
```

Result:

Analog input was successfully converted into digital data and the sensor value was obtained correctly.